

Laboratório Circuitos Combinacionais

Prof. Fernando Gehm Moraes

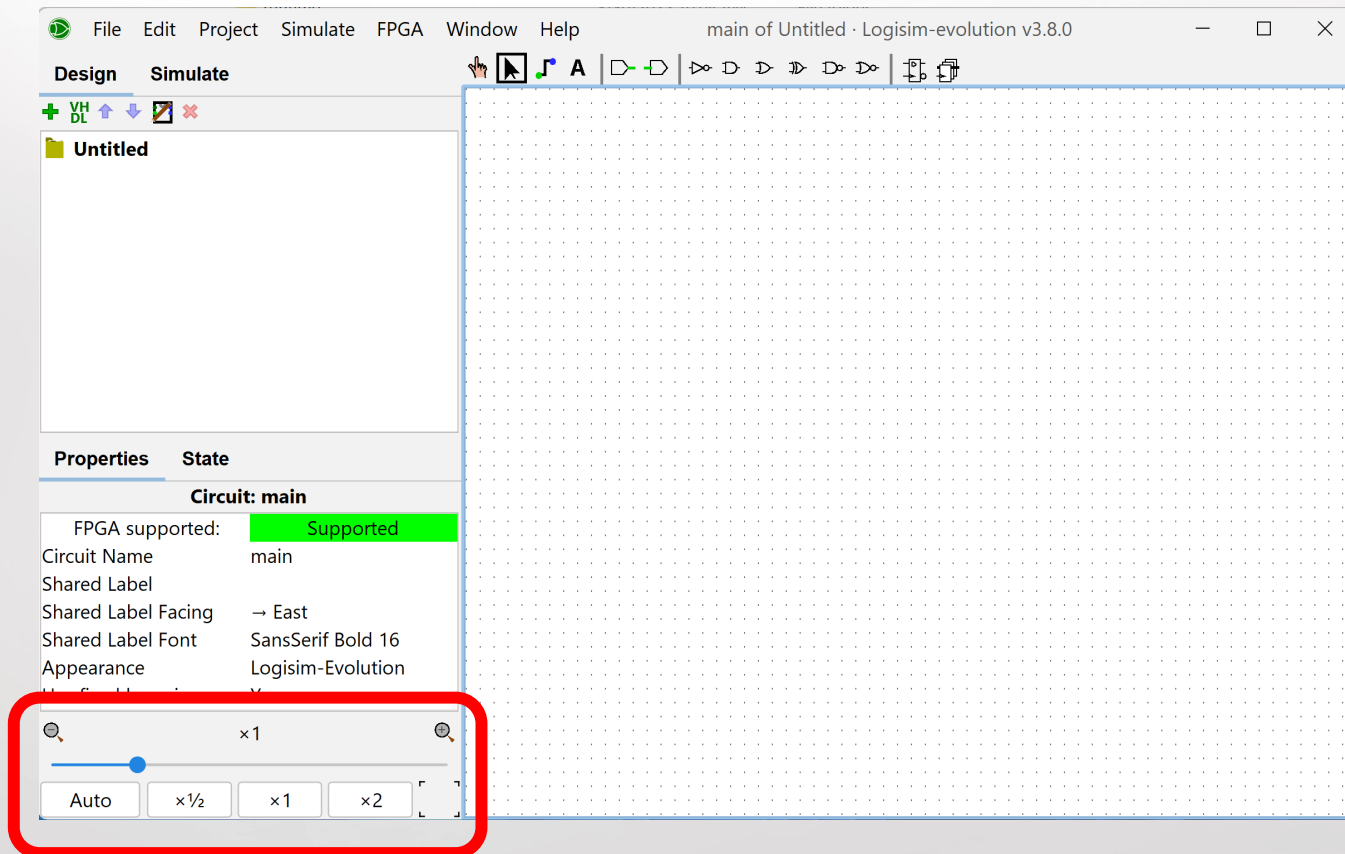


Baixar a aplicação

<https://github.com/logisim-evolution/logisim-evolution/releases/tag/v3.9.0>

<https://github.com/logisim-evolution/logisim-evolution/releases/download/v3.9.0/logisim-evolution-3.9.0-all.jar>

Tela Inicial



Zoom

PARTE I

FA: FULL ADDER

Primeiro módulo: **FA**

$$C_o = A.B + B.C_i + A.C_i$$

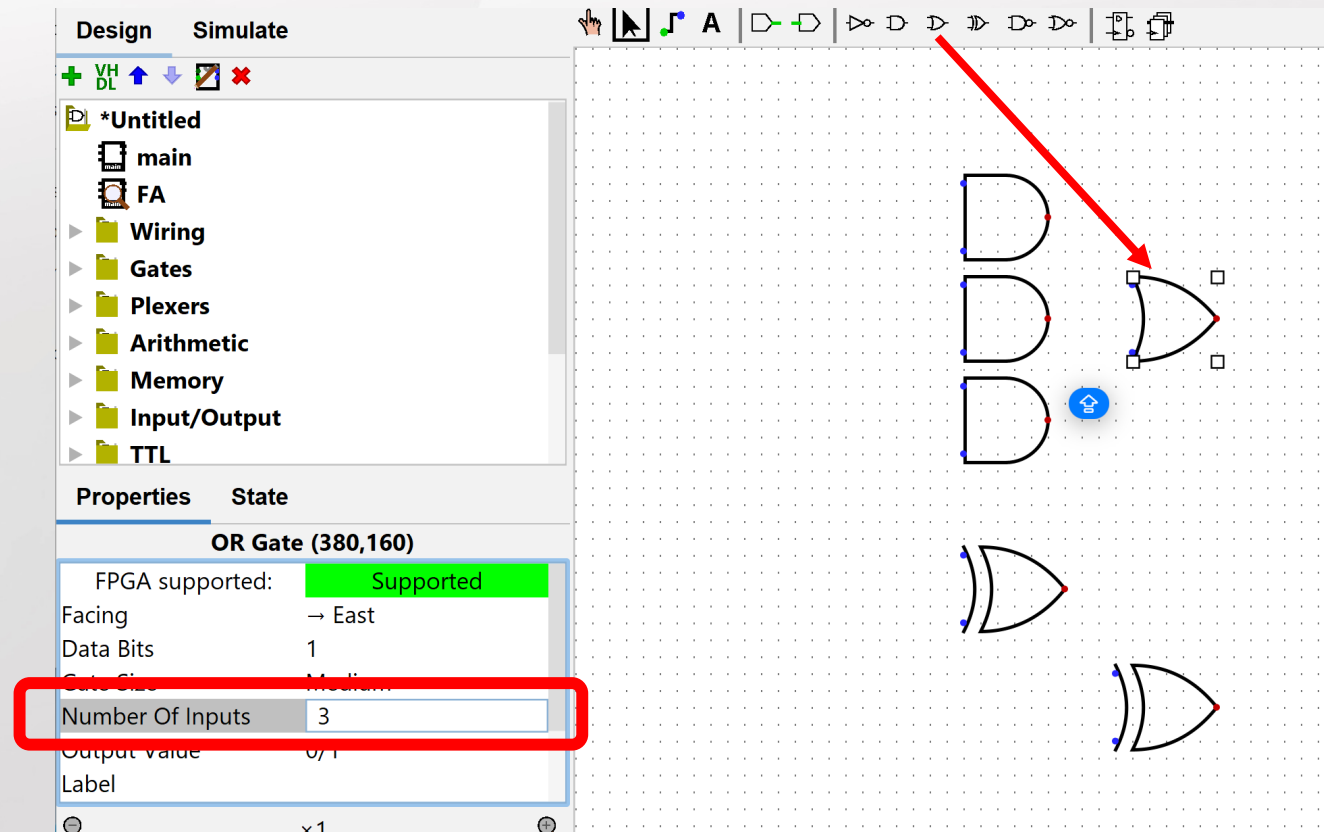
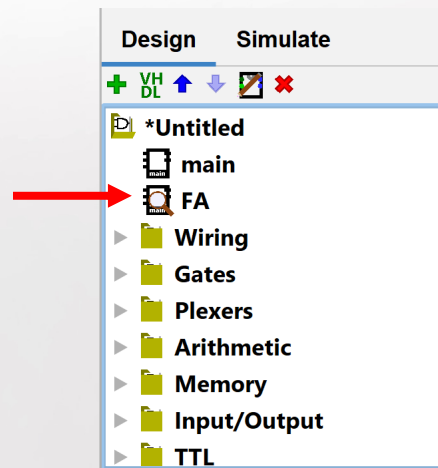
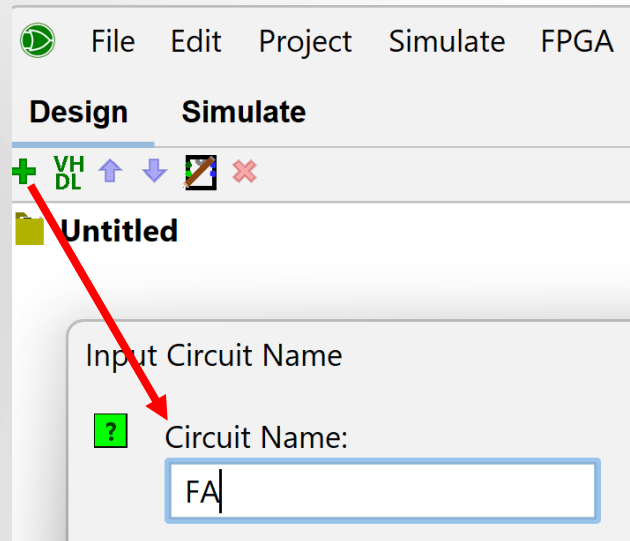
$$S = A \oplus B \oplus C_i$$

Clicar em **+** e definir o nome FA

Clicar em FA

INSERIR PORTAS LOGICAS

Clicar na porta, e depois clicar na área do circuito
Em destaque a seleção da or para 3 entradas



Primeiro módulo: FA

$$C_o = A \cdot B + B \cdot C_i + A \cdot C_i$$

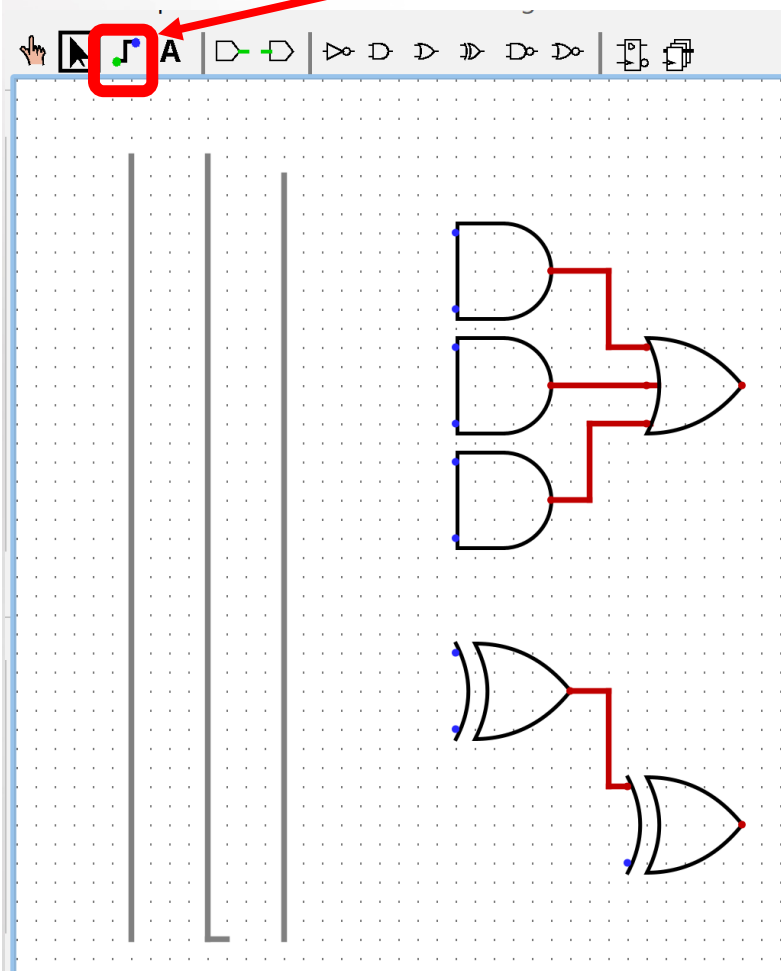
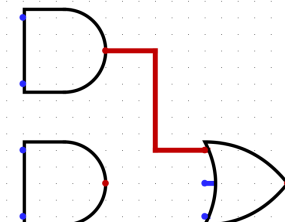
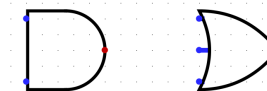
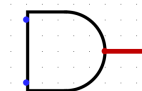
$$S = A \oplus B \oplus C_i$$

FIOS

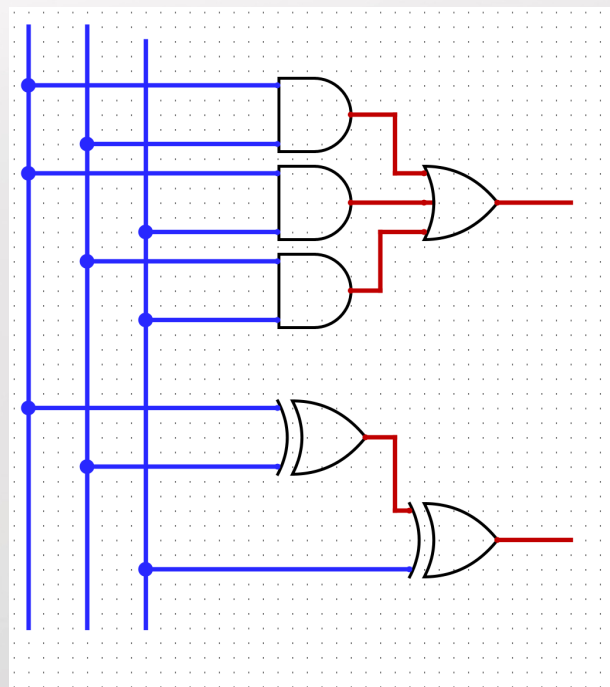
Desenhe três fios verticais para A, B, C

Lige as portas – **dica:** faça os fios por partes

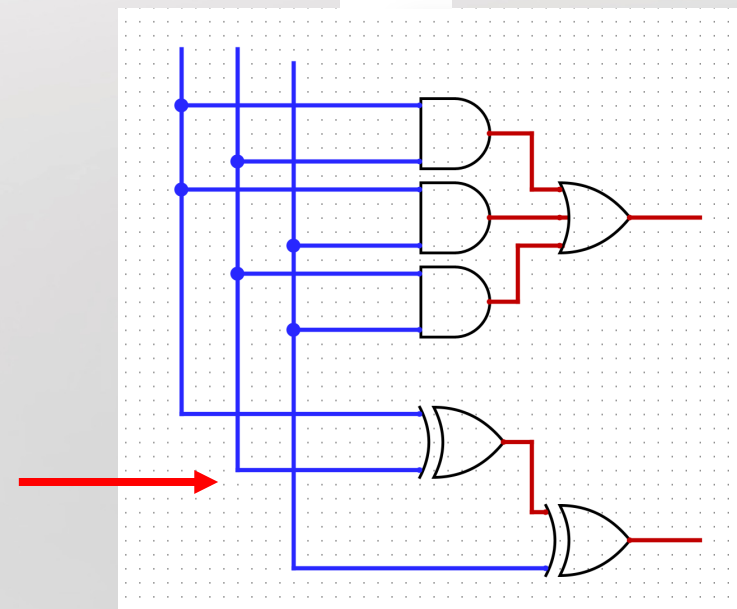
Use  para “parar” o fio



Complete as ligações



Remova os fios desnecessários (opcional) 



Primeiro módulo: **FA**

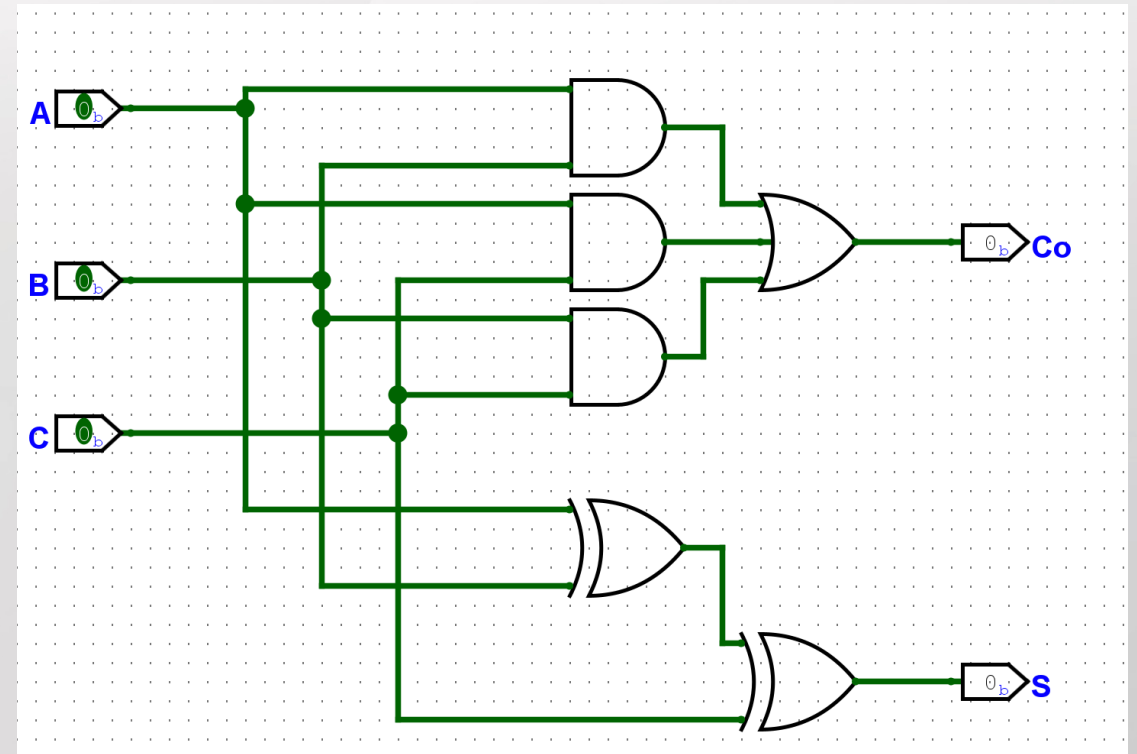
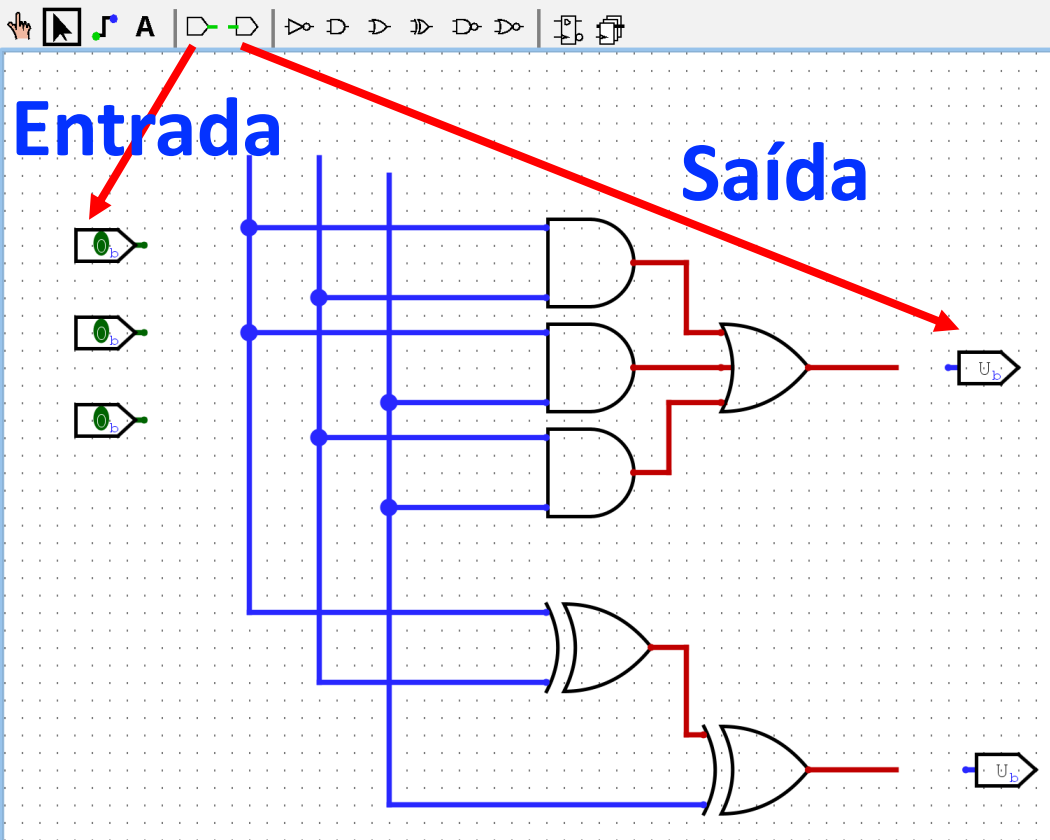
$$C_o = A.B + B.C_i + A.C_i$$

$$S = A \oplus B \oplus C_i$$

Pinos de entrada e saída

Insira os pinos para A/B/C e Co/S

Duplo clique nos pinos para nomeá-los
Terminar as conexões
(opcional) Remover fios sobrando



Primeiro módulo: **FA**

$$C_o = A.B + B.C_i + A.C_i$$

$$S = A \oplus B \oplus C_i$$

Testando



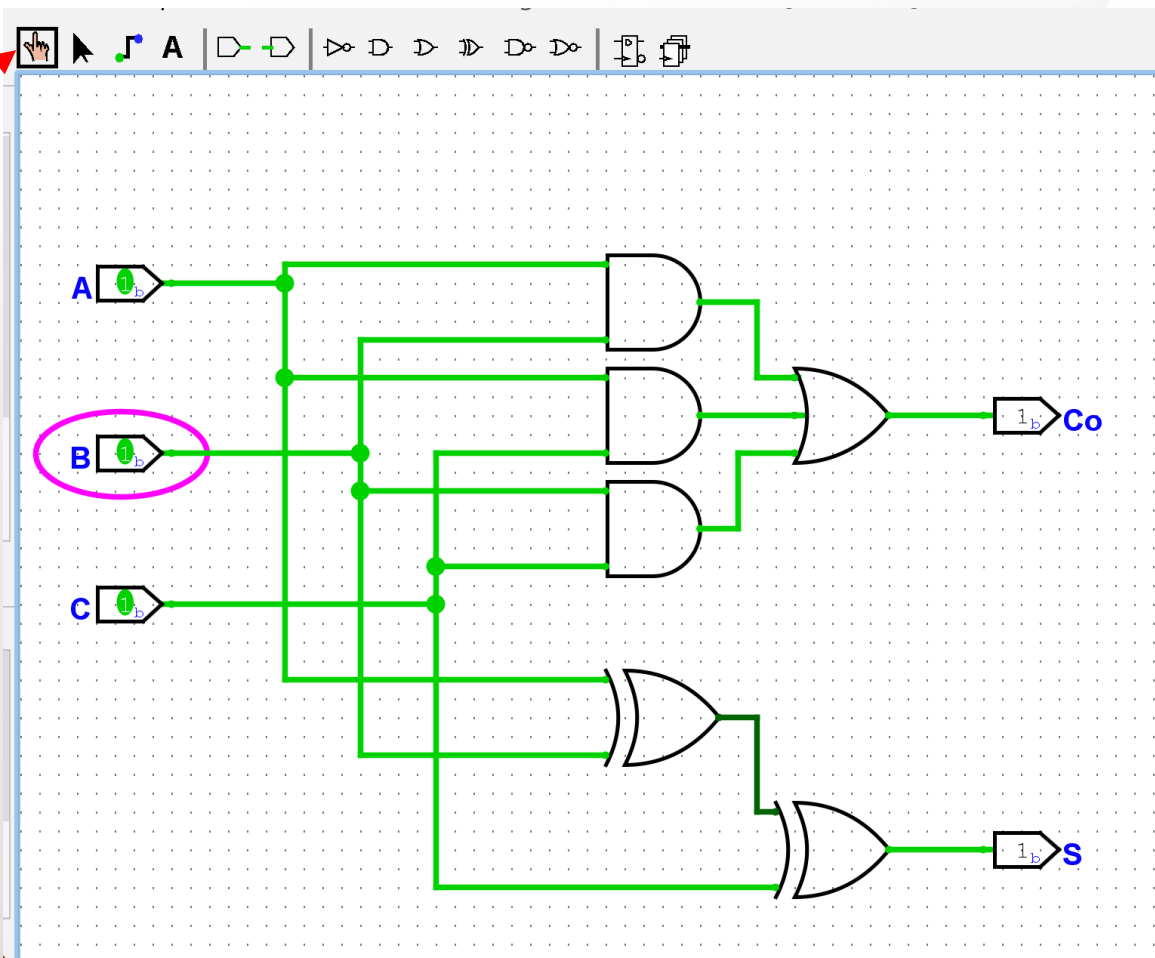
No exemplo temos

$$1 + 1 + 1$$

$$S = 1$$

$$C_o = 1$$

Testar o circuito de acordo com a tabela verdade



A	B	C _i	S	C _o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

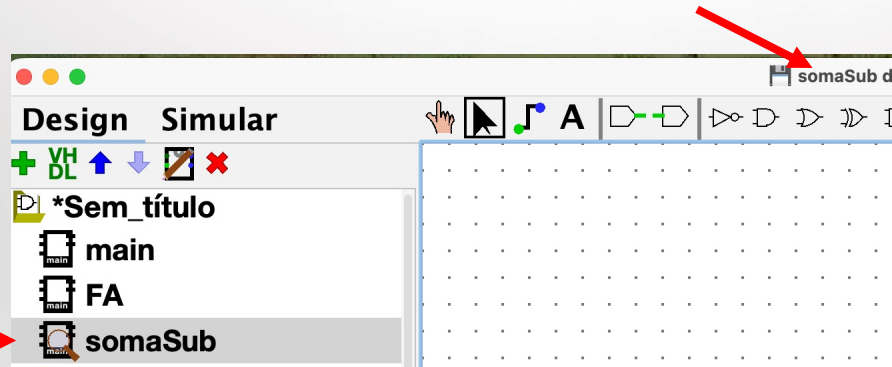
PARTE II

SomaSub: Somador/Subtrator de 4 bits

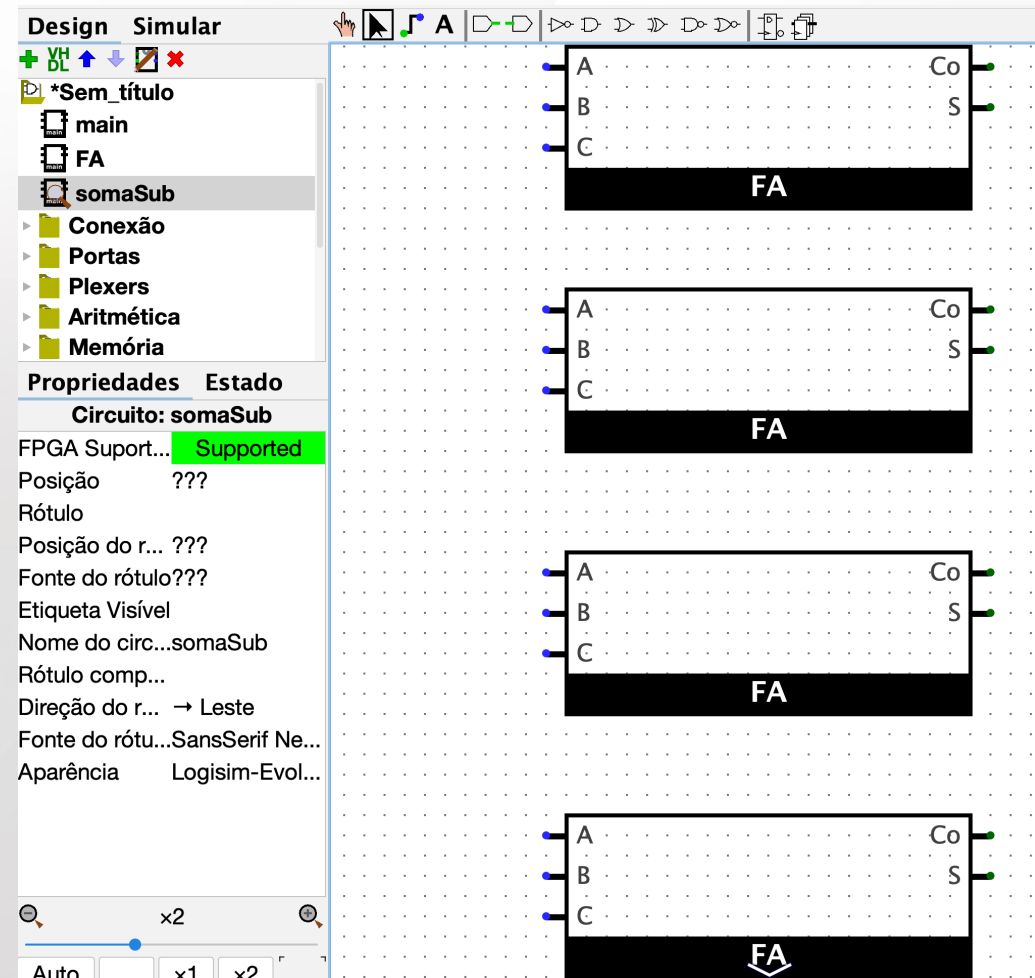
Clicar em + e definir o nome somaSub

? Nome do circuito:

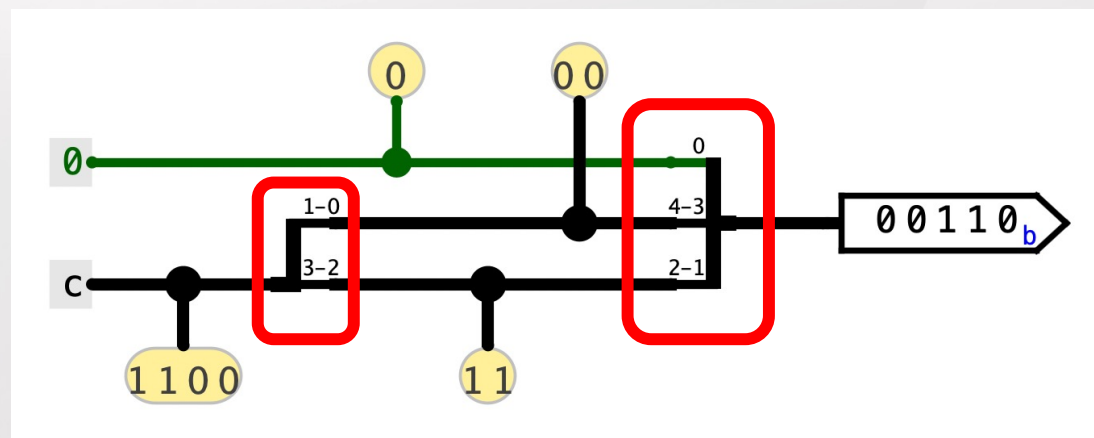
Cancelar Está bem.



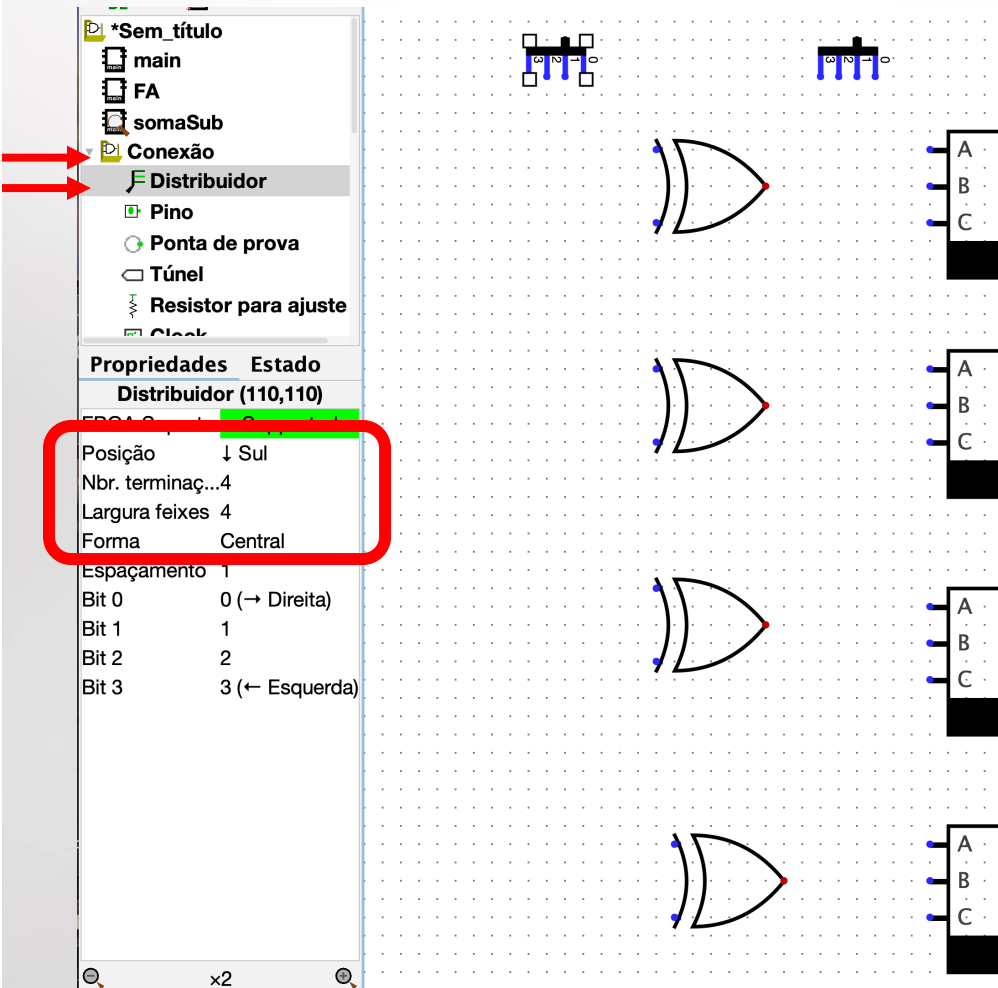
Com o Soma4 selecionado, clicar no FA e depois na área de trabalho, posicionando 4 FA



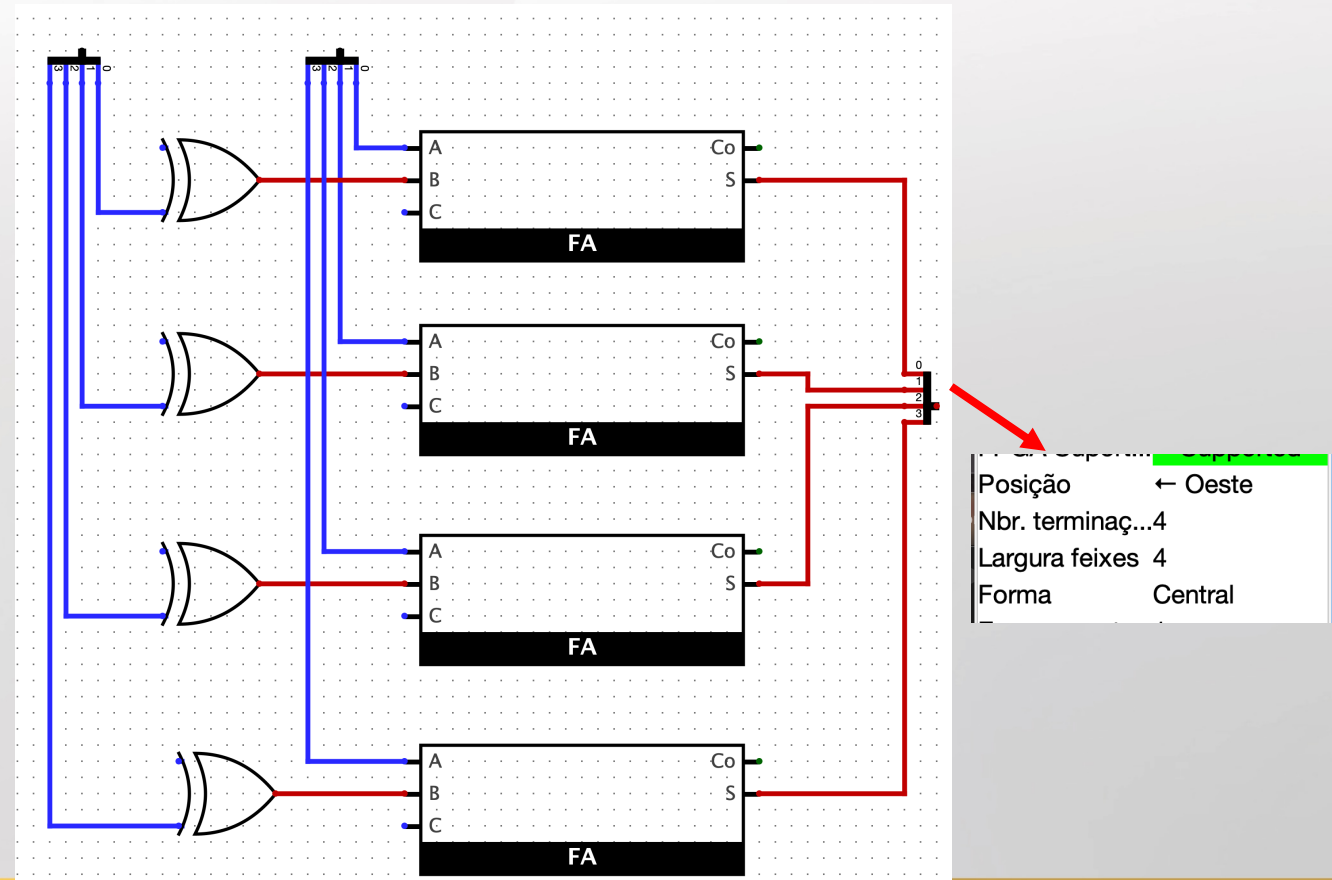
Distribuidores (*Splitters*) permitem que você pegue um valor de múltiplos bits e o divida em partes menores ou combine vários valores de um ou mais bits em um único valor. Neste exemplo, dividimos o número binário de 4 bits **1100** em **11** e **00**, invertendo suas posições e combinando-os com **0** para criar o número final de 5 bits: **00110**.



Insira dois distribuidores (entra n fios e depois separa o n fios) e 4 XOR (para negar B)
 - observar a configuração destacada



Realize as conexões como abaixo (A/B/S)
 Insira um *splitter* para a



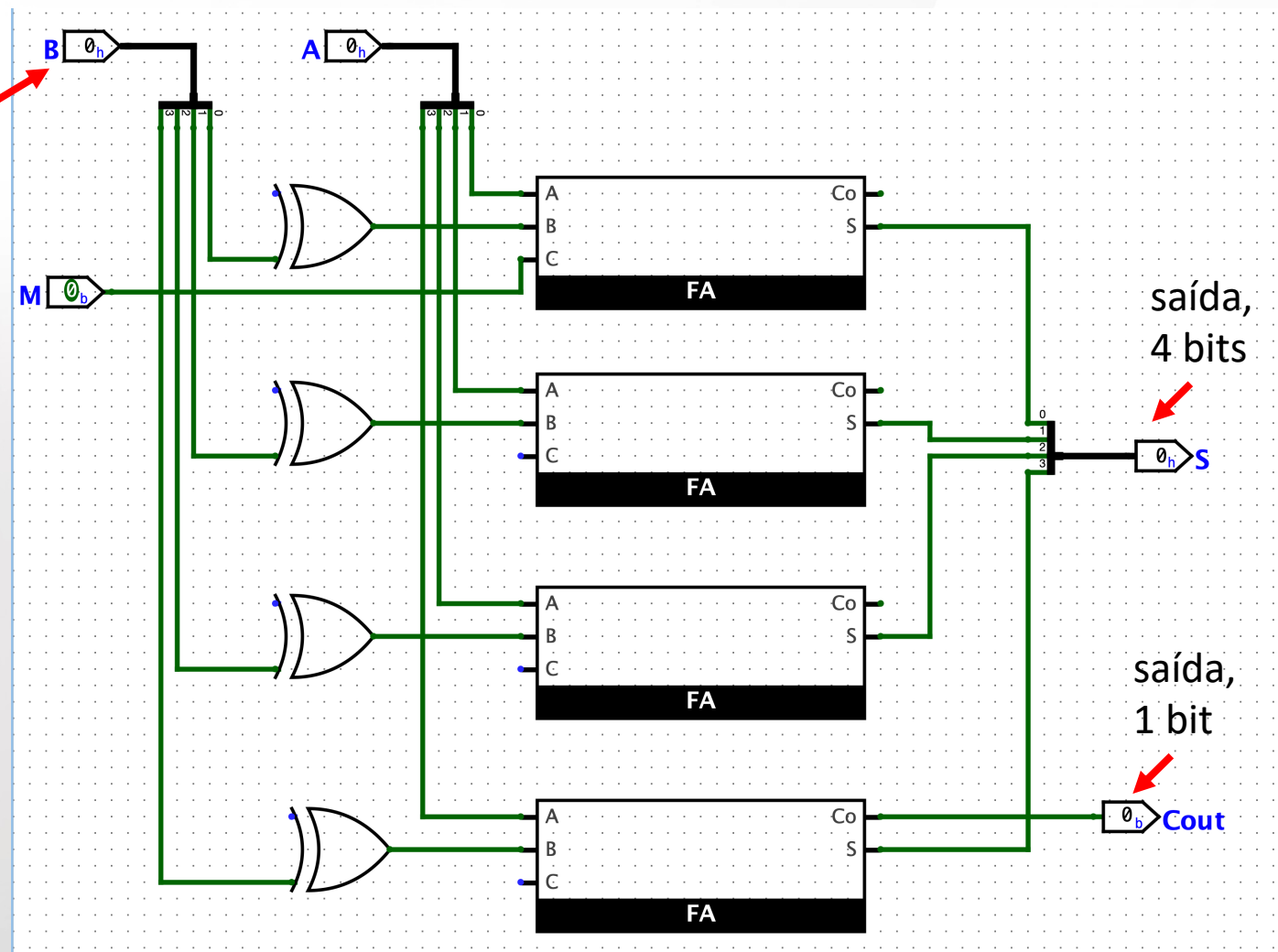
Inserir os pinos A / B / S abaixo. Estes 3 pinos são de 4 bits.
Inserir o pino M (mode) e Cout

FPGA Suport... Supported

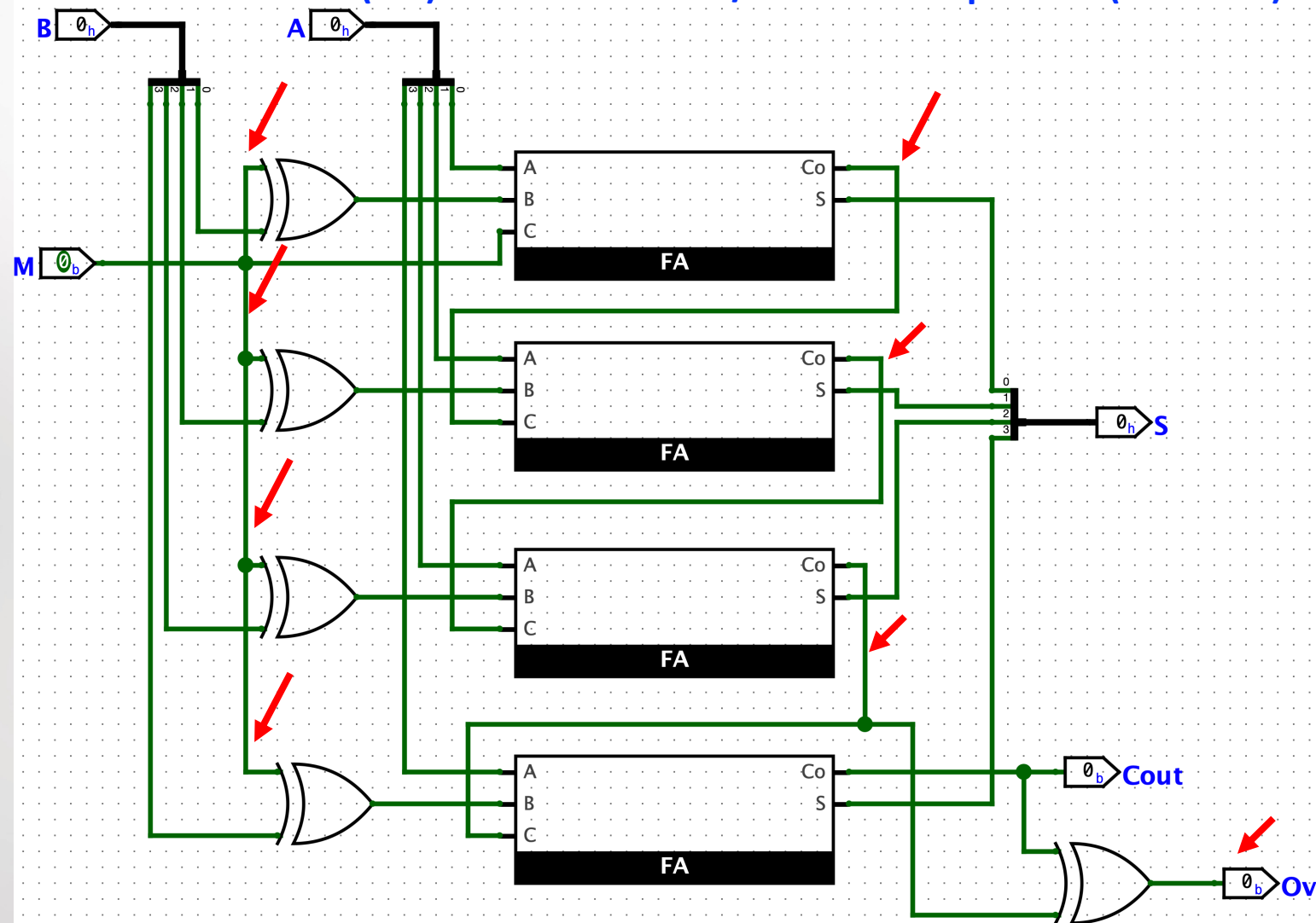
Posição → Leste

Saída? Não

Bits de dados 4

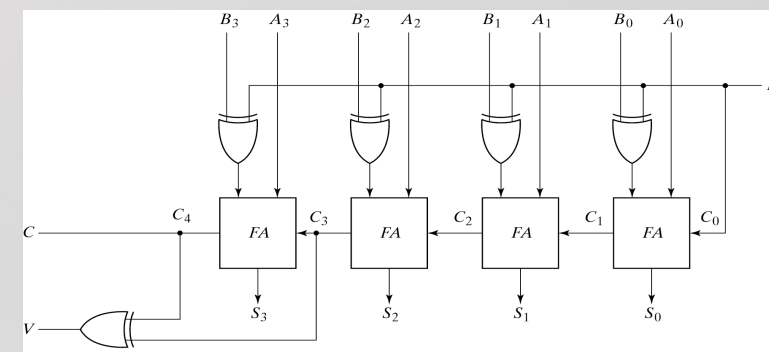


Conecte os Co nos C (Cin) como destacado / Insira a Xor para Ov (overflow) e o respectivo pino de saída

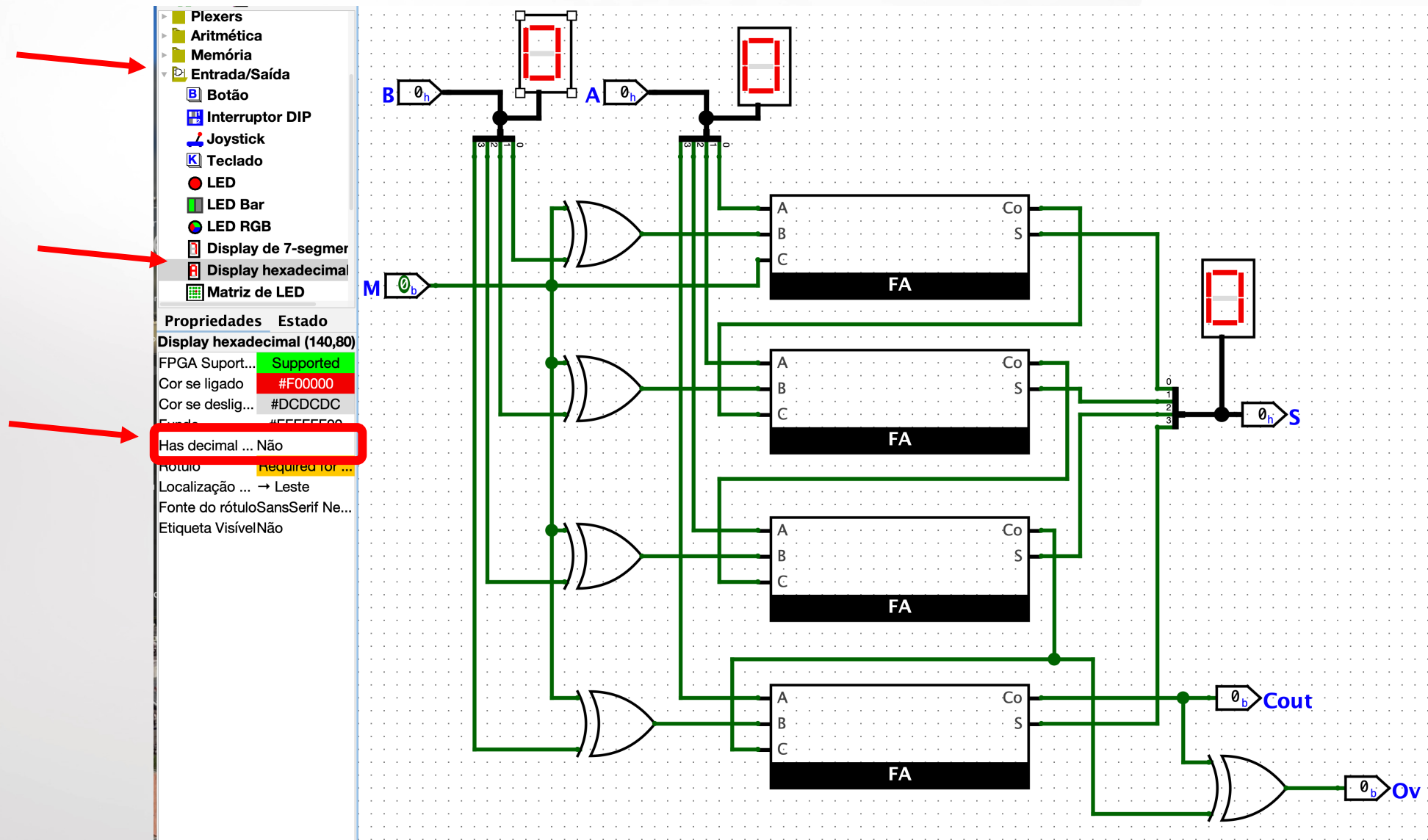


Atenção ao M (mode):
ele é conectado a todas as XOR de B
e ao primeiro C (Cin)

Circuito visto em aula:

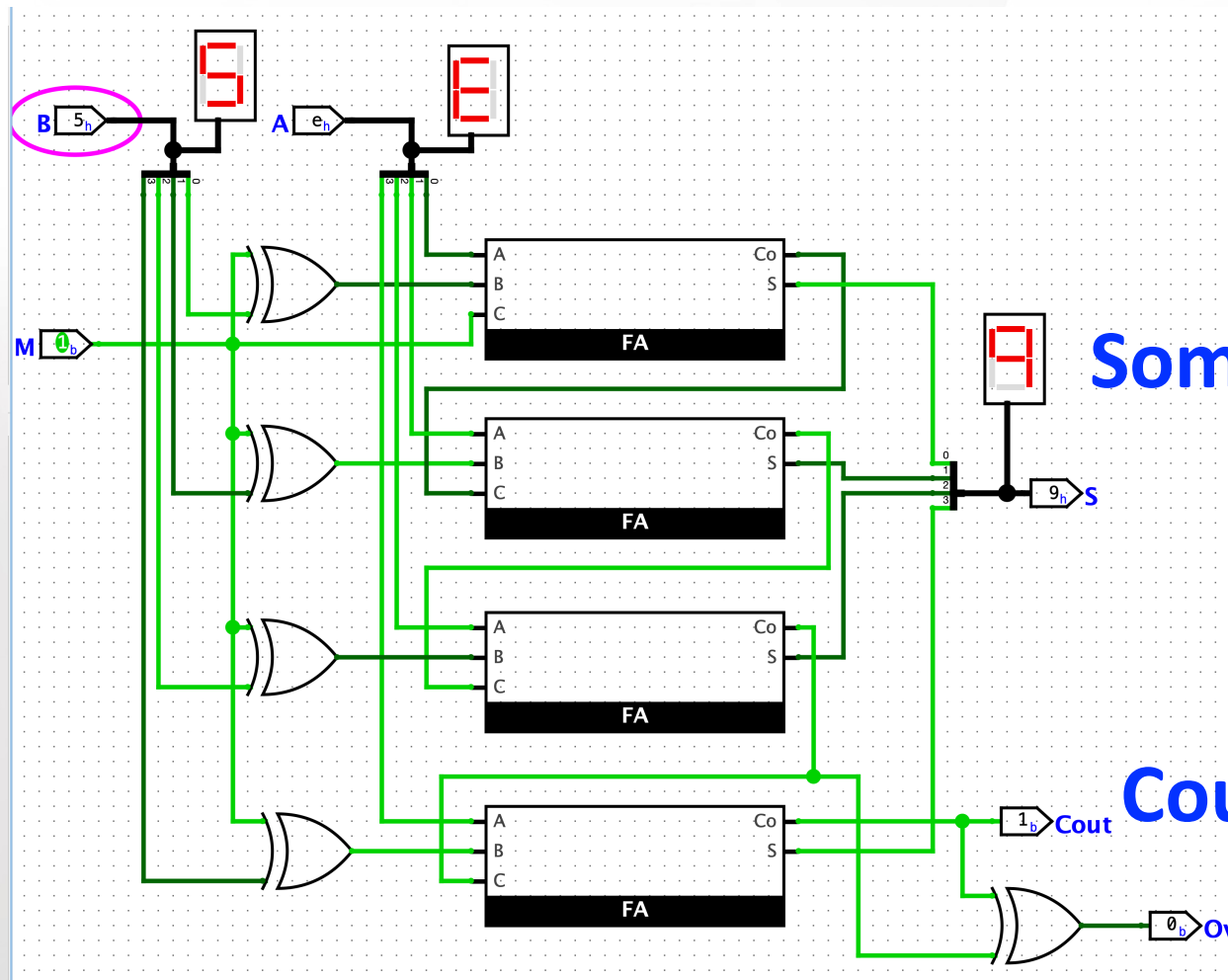
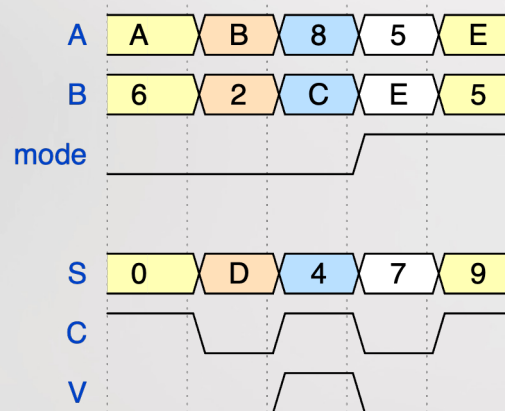


Concluir o somador inserindo 3 “hex digit Display”, sem ponto decimal, conectando-os em A / B / S



E – 5 (mode = 1)

Testar



Soma=9

Cout=1

Ov=1

PARTE III

Unidade Aritmética

Criar o módulo Uarit

Conexão

- Distribuidor
- Pino
- Ponta de prova
- Túnel
- Resistor para ajuste
- Clock
- POR
- Constante

Constante (90,330)

FPGA Suport... Supported

Posição → Leste

Bits de dados 4

Valor 0x1

Constante (120,200)

FPGA Suport... Supported

Posição → Leste

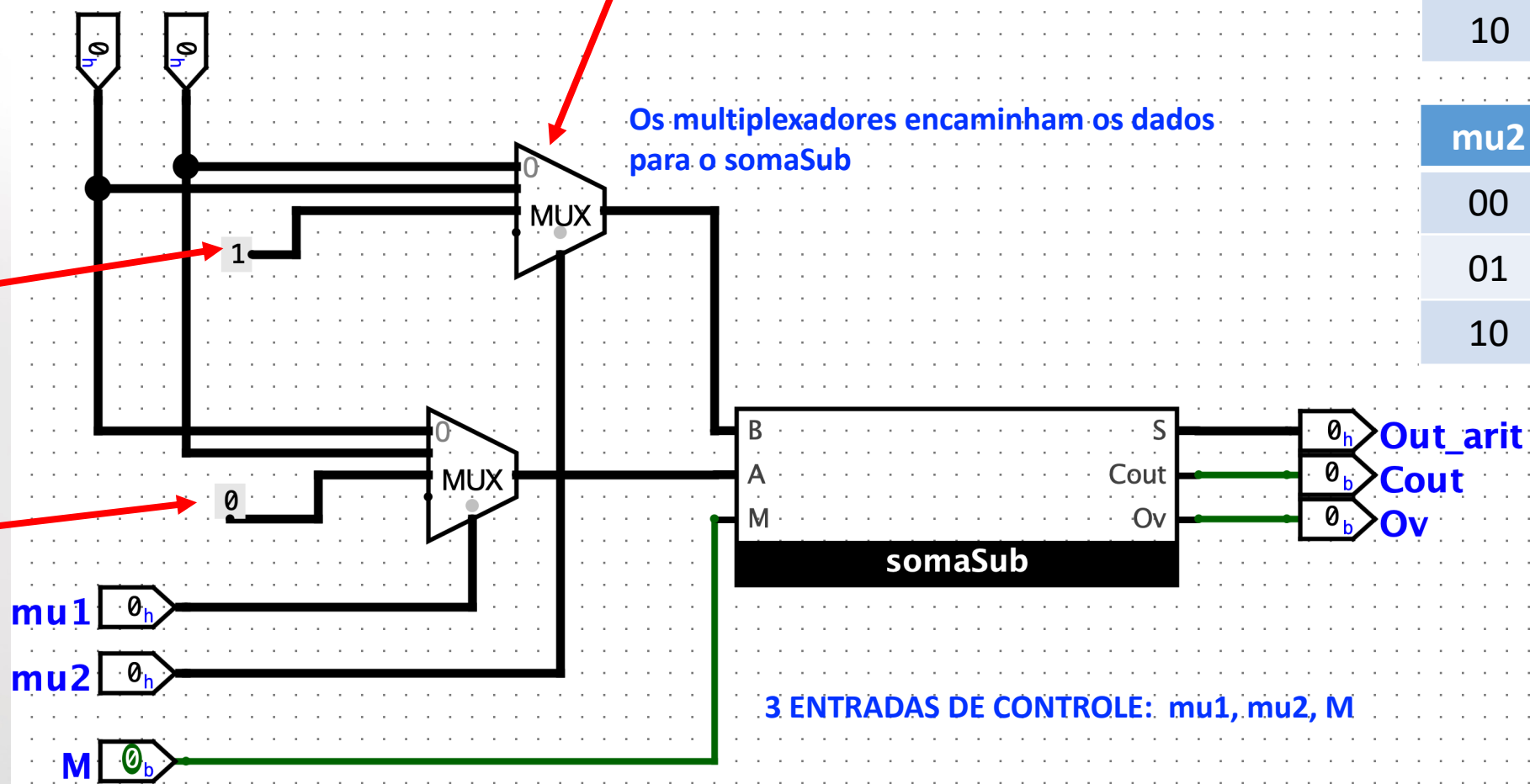
Bits de dados 4

Valor 0x0

Posição	→ Leste
Tamanho do ...	Grande
Selecionar p...	Abaixo/Esqu...
Bits para sele...	2
Bits de dados	4

OP1 e OP2
De 4 bits

OP1 OP2



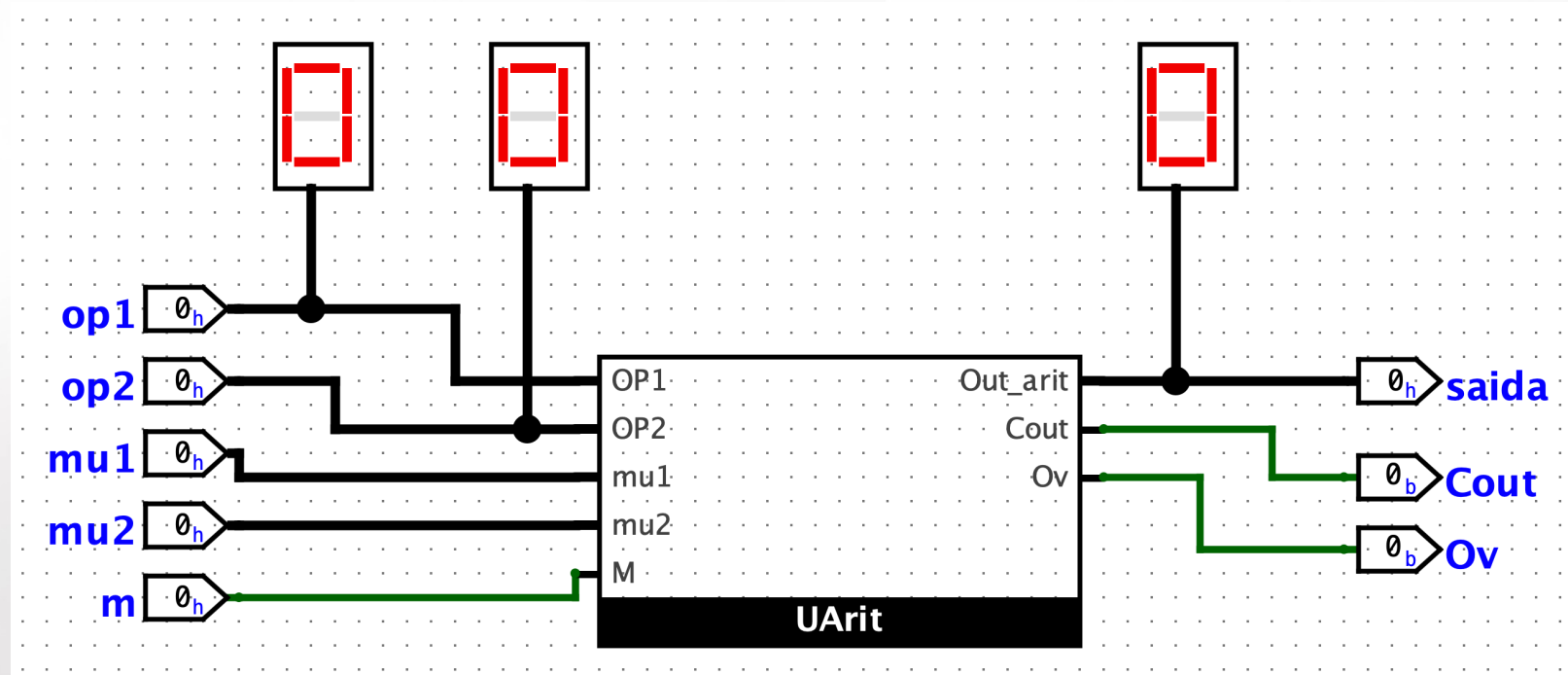
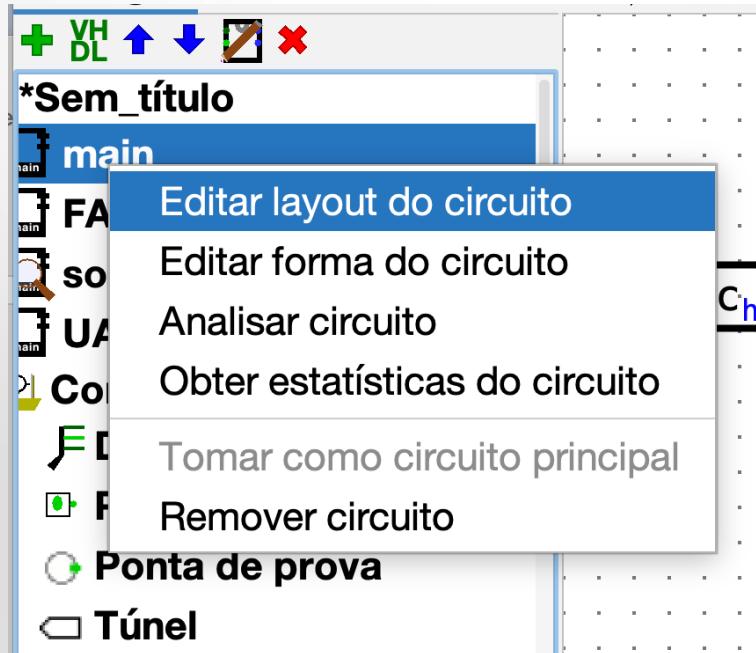
mu1	A
00	op1
01	op2
10	1

mu2	B
00	op2
01	op1
10	0

PARTE IV

Gerando o “top”

Clicar em “main” - e editar

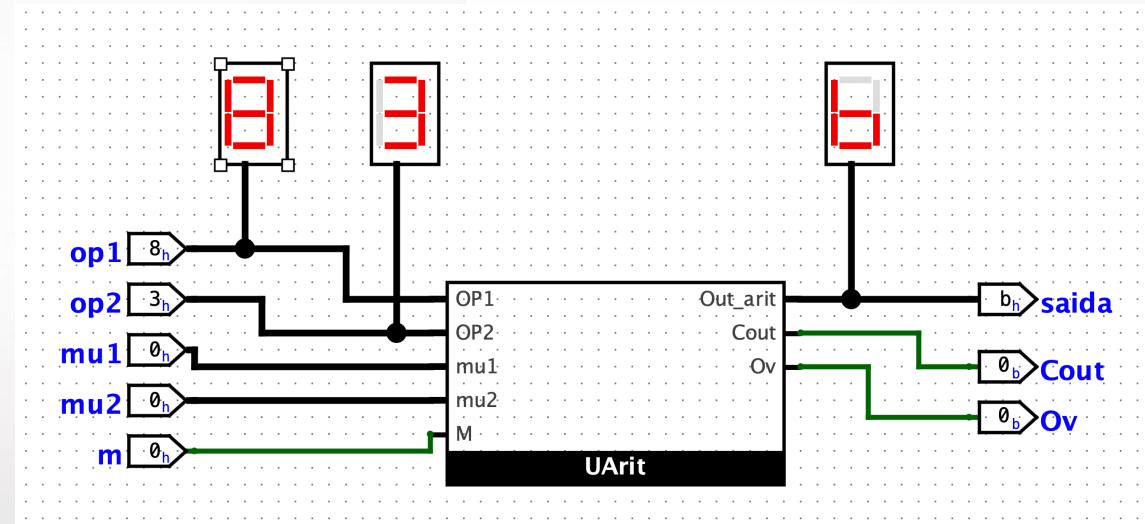


Interpretando e testando

mu1	A	mu2	B
00	op1	00	op2
01	op2	01	op1
10	1	10	0

mu1=0 / mu2=0 / m=0

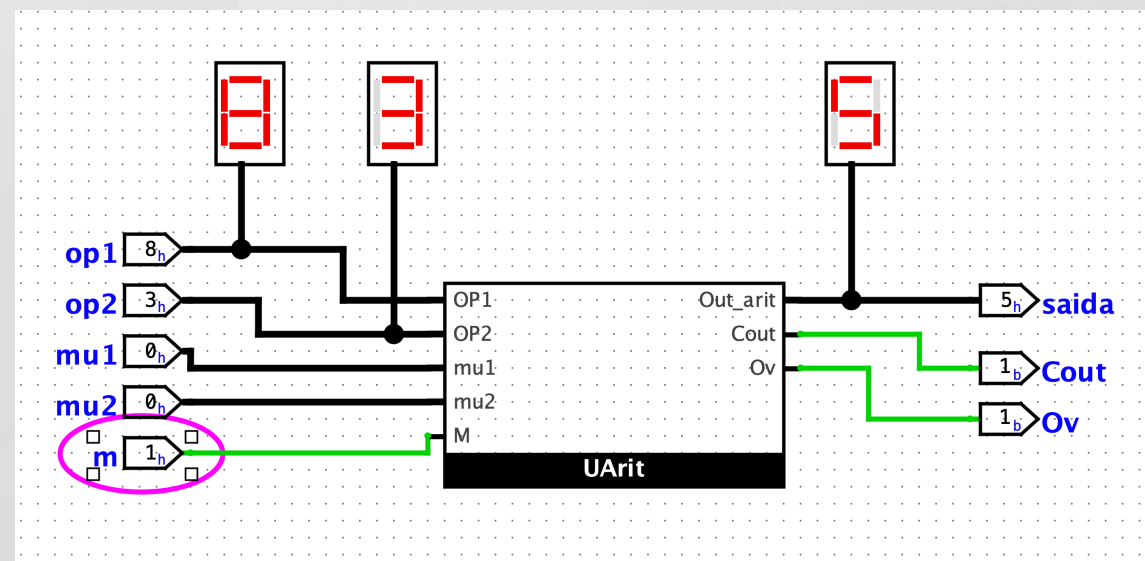
op1 + op2 → soma



mu1=0 / mu2=0 / m=1

op1 - op2 → sub

Porque ov=1?



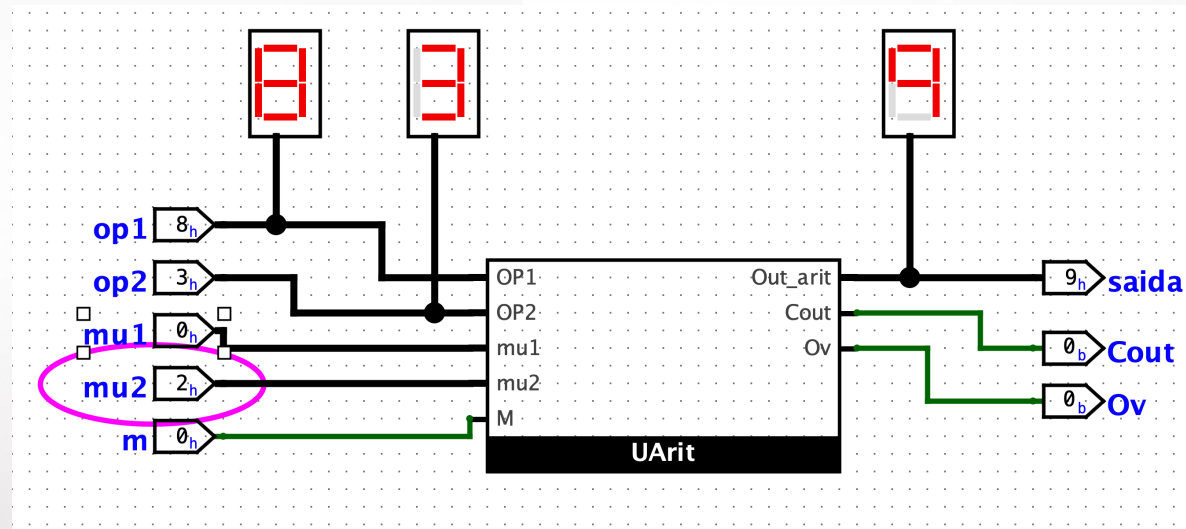
Interpretando e testando

mu1	A
00	op1
01	op2
10	0

mu2	B
00	op2
01	op1
10	1

mu1=0 / mu2=2 / m=0

1 + op1 → INC OP1

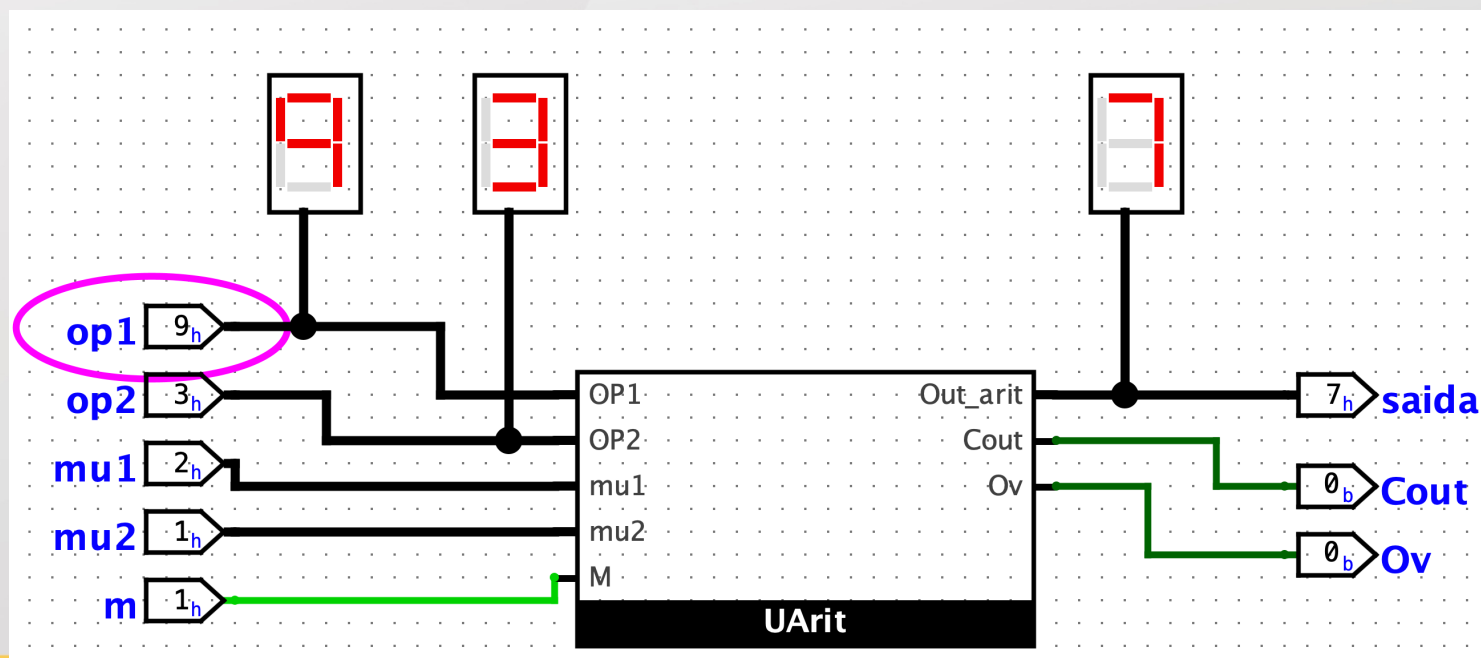


mu1=2 / mu2=1 / m=1

0 - op1 → negativo de op1

1001

Neg: 0110+1 = 0111 = (7)₁₀

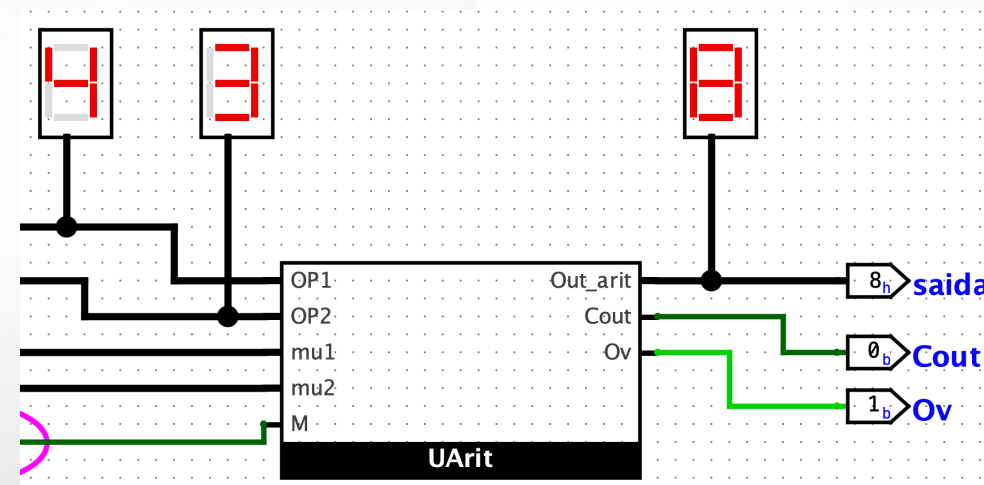


Interpretando e testando

mu1	A	mu2	B
00	op1	00	op2
01	op2	01	op1
10	0	10	1

mu1=? / mu2=? / m=?

2OP1



mu1=? / mu2=? / m=?

op2 – op1

